

CS202 · DESIGN PATTERNS

Observer Design Pattern

Notify many objects — without letting the source know who they are.

Nguyễn Đình Minh Huy — 25125083

Trần Gia Huy — 25125084

APCS 2025 · Group 52



WHAT WE WILL COVER

Agenda

01

Real-world problem

02

Naïve solution & drawbacks

03

The Observer pattern

04

Class diagrams

05

Code implementation & demo

06

Pairing with Mediator & Interpreter

07

Pros & cons

08

Quiz

09

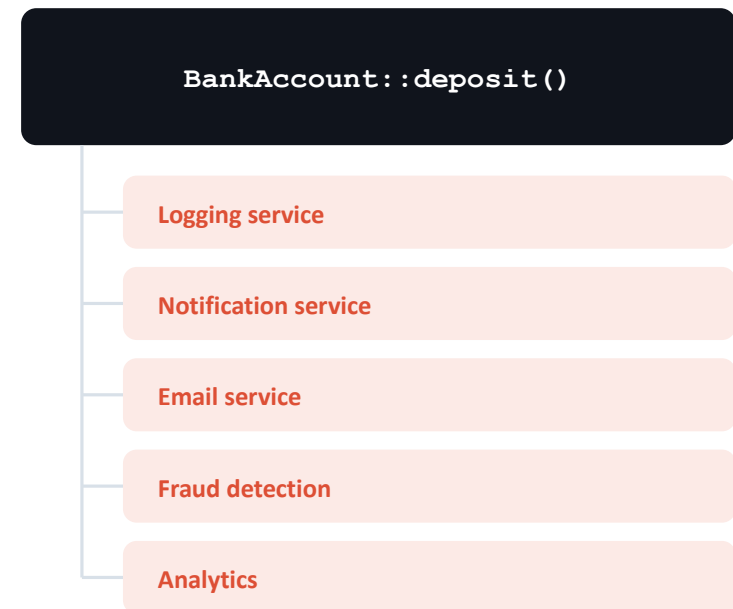
Conclusion

THE SETTING

Real-world problem

We are building a bank management system: it stores users' accounts and lets them deposit money into those accounts.

However, when deposit is called, aside from adding money we are required to run other procedures — logging, notifying, emailing, fraud checks. These belong to other services that we do not fully control.



Five extra services — five hard-wired calls.

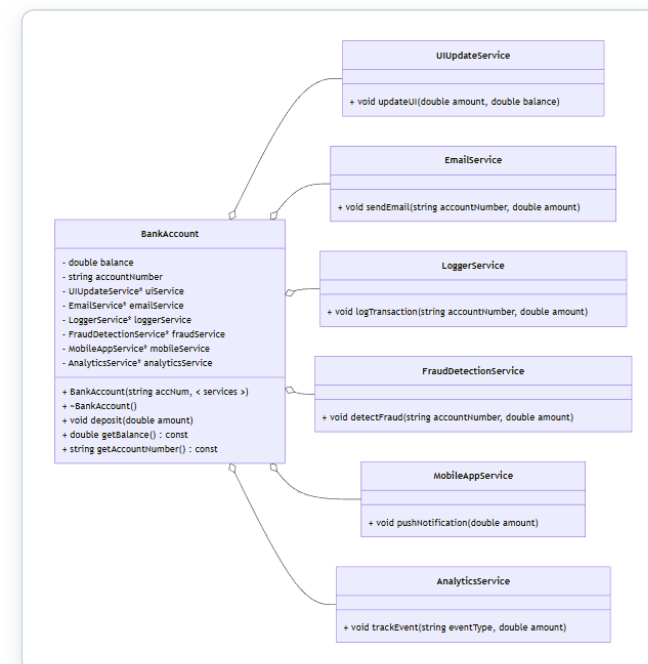
STEP 1 · THE OBVIOUS WAY

Naïve solution

When `BankAccount::deposit` is called, it invokes every required service directly — each one is stored as a pointer inside the account.

```
ui.update(balance);  
email.sendEmail(amount);  
logger.log(amount, balance);  
fraudDetector.scan(amount);  
mobileApp.pushNotification(amount);  
analytics.trackActivity("Deposit", amount);
```

BankAccount must know all six concrete types and the exact name of every method it calls.



Naïve BankAccount UML

Naïve solution — drawbacks



Tight coupling

BankAccount must know the exact concrete types of the services and their specific update methods.



Violates the Open–Closed Principle

Changing or adding a service forces us to reopen and modify BankAccount itself.



Rigid and hard to scale

Every change to a service — or to the account — ripples through the other side.

THE INSIGHT

Two observations

01

The services take similar — but not identical — parameters. With one general structure, BankAccount no longer needs to know what each service requires.

02

BankAccount never uses the results of those calls. It does not care who runs; it only needs a list of things to call when depositing.

⇒ Observer pattern

One list of subscribers. One shared interface. One call to notify them all.

THE PATTERN

Observer — the intuition

Imagine a popular YouTube channel, or a digital newsletter.

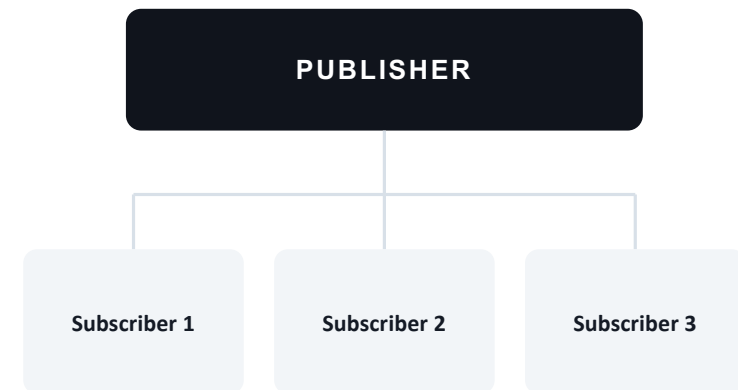
Subscribers should not have to keep asking “is there a new video?” or “is there a new edition?”

The publisher maintains the list of subscribers. When new content appears, it pushes a notification to everyone on that list.

Tracking state changes moves from the consumers to the source — instant updates, no wasteful polling.

In the Observer pattern:

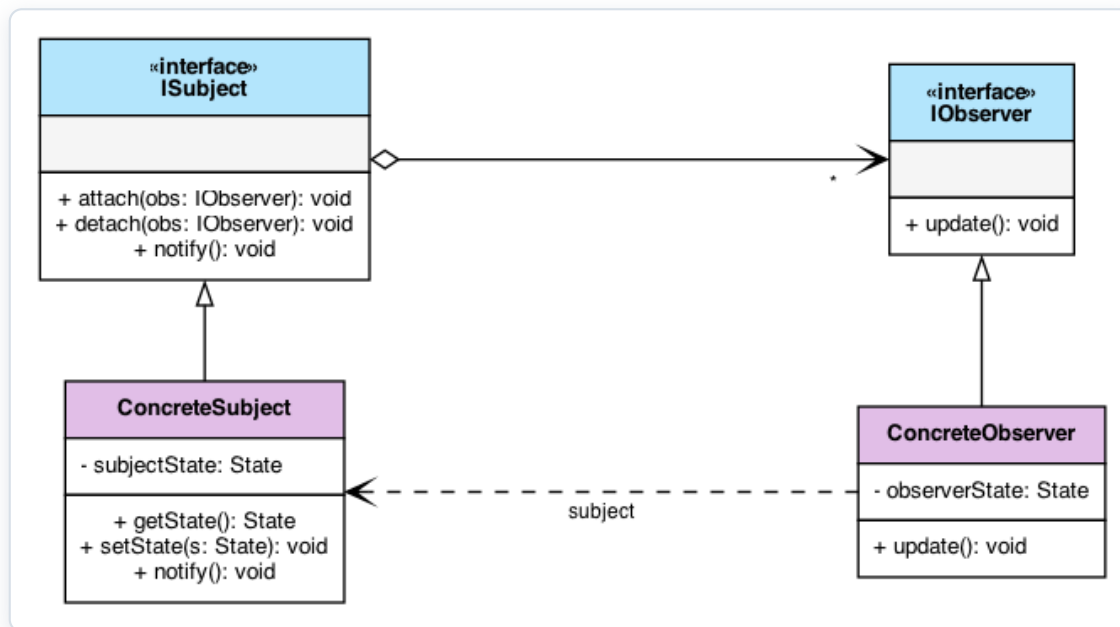
Subject = Publisher · **Observer = Subscriber**



one notify() → every subscriber updated

STRUCTURE

General class diagram



The Gang-of-Four structure: ISubject and IObserver

ISubject

Maintains the list of observers. Implements `attach()` and `detach()`, and broadcasts `notify()` whenever its state changes.

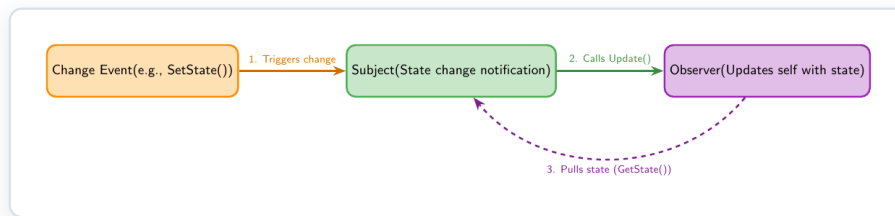
IObserver

Exposes a single `update()` method that the subject calls during a notification.

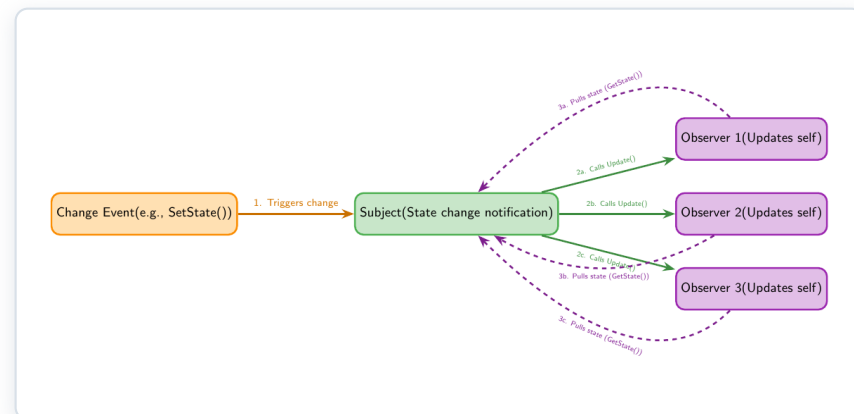
STRUCTURE

How a notification flows

1 SUBJECT · 1 OBSERVER



1 SUBJECT · MANY OBSERVERS



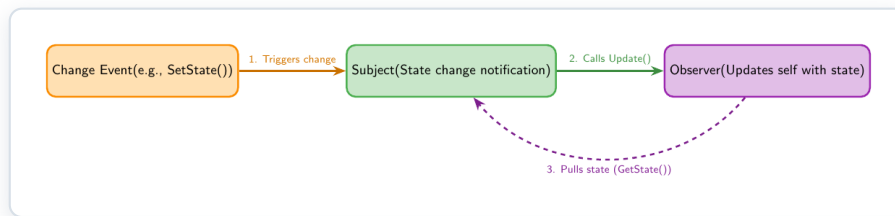
PULL MODEL

The subject only announces that something changed; each observer then retrieves the data it needs through a getter.

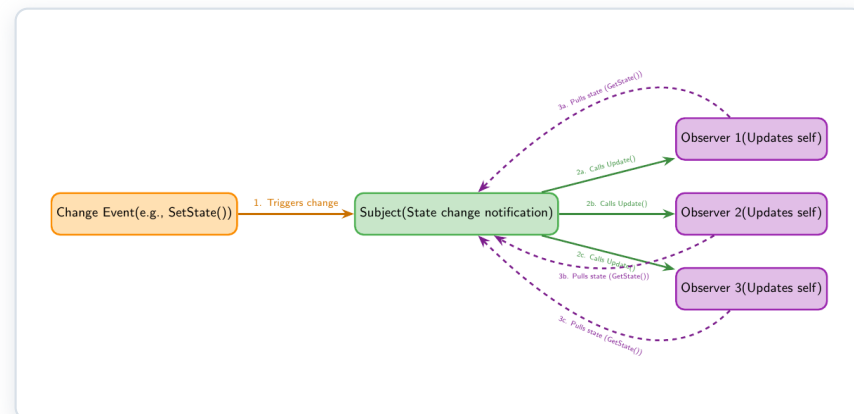
STRUCTURE

How a notification flows

1 SUBJECT · 1 OBSERVER



1 SUBJECT · MANY OBSERVERS



PULL MODEL

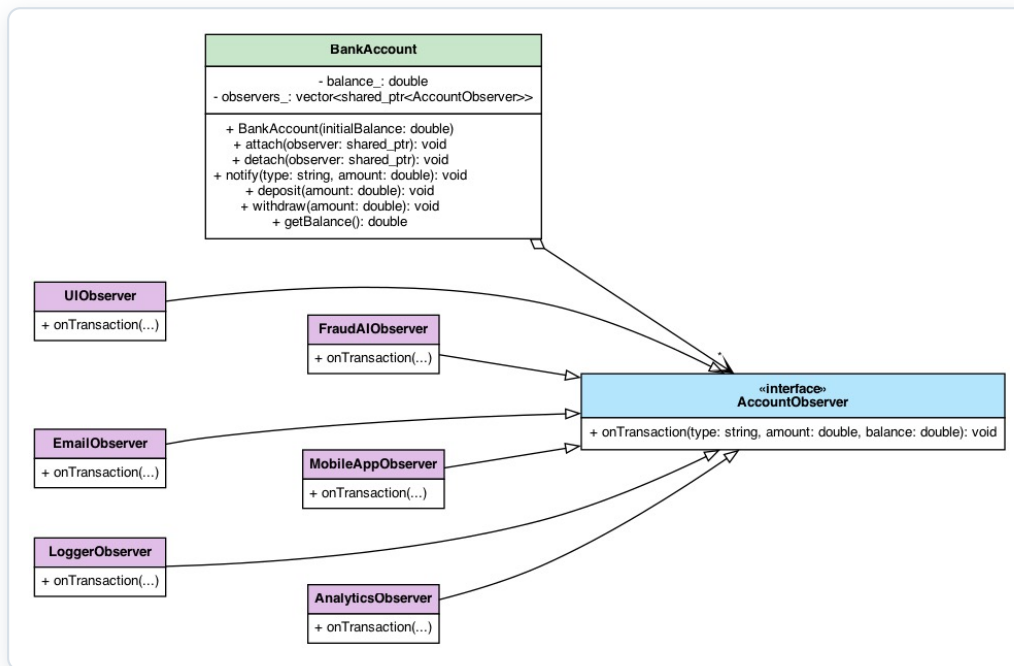
The subject only announces that something changed; each observer then retrieves the data it needs through a getter.

PUSH MODEL

The alternative: the subject sends the changed data directly along with the notification.

CASE STUDY

Bank account — class diagram



BankAccount is the subject; every service becomes an AccountObserver

Concrete observers

UIObserver

EmailObserver

LoggerObserver

FraudAIObserver

MobileAppObserver

AnalyticsObserver

All six implement one interface: `onTransaction(type, amount, balance)`.

CASE STUDY

What the pattern buys us



True isolation

BankAccount is completely ignorant of the concrete services — it only knows the observer interface.



Open/Closed Principle

New services can be added without modifying the BankAccount class.



Single Responsibility

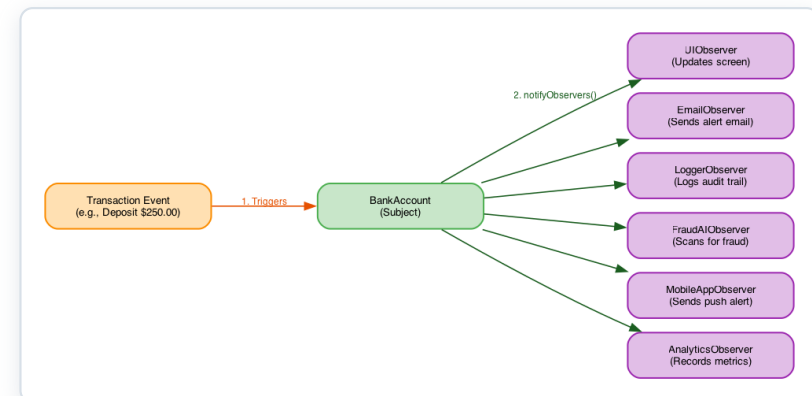
Core financial logic stays clean; logging, fraud checks and UI live in dedicated components.



Runtime flexibility

Observers can be attached or detached on the fly, based on user settings or system state.

ONE CALL, SIX REACTIONS



A single transaction event travels through `BankAccount::notify()` and reaches every subscribed observer — the account never names one of them.

Naïve approach — implementation

```
naive.hpp - Naive Bank Account Implementation

// Naive Bank Account Implementation (Observer/src/naive.cpp)
#include <iostream>
#include <string>

// Tightly-coupled Naive Bank Account
class NaiveBankAccount {
private:
    double balance;
    UI ui;
    EmailSystem email;
    Logger logger;
    FraudDetector fraudDetector;
    MobileApp mobileApp;
    Analytics analytics;

public:
    NaiveBankAccount(double initialBalance = 0.0)
        : balance(initialBalance) {}

    void deposit(double amount) {
        balance += amount;

        // Direct, coupled method invocations on concrete systems
        ui.update(balance);
        email.sendEmail(amount);
        logger.log(amount, balance);
        fraudDetector.scan(amount);
        mobileApp.pushNotification(amount);
        analytics.trackActivity("Deposit", amount);
    }
};
```

Observer/src/naive.cpp

HARD-WIRED

The account owns a member for every service: UI, EmailSystem, Logger, FraudDetector, MobileApp, Analytics — and deposit() calls each of them by name.

CONSEQUENCE

A service cannot be detached at runtime, and a new one cannot be added without editing this file.

Naïve approach — demo

```
Terminal - bank_naive_demo

=====
BANK ACCOUNT DEMO: NAIVE VERSION
=====
Step 1: Initializing NaiveBankAccount with $1000.00...

Step 2: Depositing $250.00. Expecting notifications to ALL 6 systems...

[UI] Balance display updated to $1250.00
[Email] Alert sent: A deposit of $250.00 was made.
[Logger] Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI] Scan complete: Deposit of $250.00 is marked OK.
[Mobile App] Push: Account credited with $250.00
[Analytics] Metric recorded: Event [Deposit] for $250.00

Step 3: Attempting to detach Email and Mobile App...
Error: Cannot detach components. NaiveBankAccount is tightly coupled
to concrete notification systems.
=====
```

Terminal - bank_naive_demo

- 1 Initialise the account with \$1000.00
- 2 Deposit \$250.00 — all six systems fire
- 3 Try to detach Email and Mobile App

Error: cannot detach components. The naïve account is tightly coupled to concrete notification systems.

Observer pattern — interface header

```
pattern.hpp - Observer Pattern Interface Header

// Core Observer Pattern Header Definition (Observer/src/pattern.cpp)
#include <iostream>
#include <memory>
#include <vector>
#include <string>

// Abstract Observer Interface
class AccountObserver {
public:
    virtual ~AccountObserver() = default;
    virtual void onTransaction(const std::string &type,
                              double amount,
                              double currentBalance) = 0;
};

// Subject Class Interface
class BankAccount {
public:
    BankAccount(double initialBalance = 0.0);

    void attach(std::shared_ptr<AccountObserver> observer);
    void detach(std::shared_ptr<AccountObserver> observer);
    void notify(const std::string &type, double amount);

    void deposit(double amount);
    void withdraw(double amount);
    double getBalance() const;

private:
    double balance;
    std::vector<std::shared_ptr<AccountObserver>> observers_;
};
```

Observer/src/pattern.cpp — pattern.hpp

AccountObserver

A pure-virtual onTransaction(type, amount, currentBalance) — the only thing the account knows about.

BankAccount

attach(), detach(), notify(), plus deposit() and withdraw() over a vector of shared_ptr observers.

Observer pattern — concrete observers

```
observers.hpp - Concrete Observers Definitions

// Concrete Observers Implementation (Observer/src/pattern.cpp)
class UIObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                      double currentBalance) override {
        std::cout << "[UI Observer] Balance updated to $" << currentBalance << "\n";
    }
};

class EmailObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                      double currentBalance) override {
        std::cout << "[Email Observer] Alert sent for " << type << " of $" << amount << "\n";
    }
};

class LoggerObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                      double currentBalance) override {
        std::cout << "[Logger Observer] Logged: " << type << " of $" << amount << "\n";
    }
};

class FraudAIObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                      double currentBalance) override {
        if (amount > 10000.0)
            std::cout << "[Fraud AI] WARNING: Suspicious activity for $" << amount << "\n";
    }
};

class MobileAppObserver : public AccountObserver { /* Push notification delivery */ };
class AnalyticsObserver : public AccountObserver { /* Activity metrics recording */ };
```

Each observer overrides onTransaction() and reacts on its own terms — the account is never edited again.

Observer pattern — demo, part 1

```
bank_observer_pattern_demo (Part 1)

=====
BANK ACCOUNT DEMO: PATTERN VERSION
=====
=== RUNNING PATTERN DEMO (With Observer Pattern) ===
Step 1: Initializing BankAccount with $1000.00...

Step 2: Attaching observers...
- UIObserver (updates screen balance display)
- EmailObserver (sends transaction notification emails)
- LoggerObserver (writes to security/audit logs)
- FraudAIObserver (scans transaction amounts, warns if > $10,000)
- MobileAppObserver (sends push notification)
- AnalyticsObserver (records activity metrics)

Step 3: Depositing $250.00. Expecting notifications to ALL 6 systems...

[BankAccount] Deposited $250.00. New balance: $1250.00
[UI Observer] Balance display updated to $1250.00
[Email Observer] Alert sent: A deposit of $250.00 was made.
[Logger Observer] Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI] Scan complete: Deposit of $250.00 is marked OK.
[Mobile App] Push: Account credited with $250.00
[Analytics] Metric recorded: Event [Deposit] for $250.00

Step 4: Dynamically detaching Email and Mobile App observers...
(Customer disabled Email alerts and Mobile pushes in settings)
=====
```

Terminal — bank_observer_pattern_demo (part 1)

1

Initialise BankAccount with \$1000.00

2

Attach all six observers

3

Deposit \$250.00 — every observer reacts

4

Detach Email and Mobile App at runtime

Same deposit() body — the subscriber list changed, not the account.

Observer pattern — demo, part 2

```

bank_observer_pattern_demo (Part 2)

=====
BANK ACCOUNT DEMO: PATTERN VERSION
=====
Step 5: Depositing $12000.00. Expecting notifications to remaining systems...
(No Email, No Mobile push. Fraud AI alert should trigger for large amount)

[BankAccount] Deposited $12000.00. New balance: $13250.00
[UI Observer] Balance display updated to $13250.00
[Logger Observer] Transaction logged: Deposit of $12000.00, new balance: $13250.00
[Fraud AI] WARNING: Suspicious large activity detected: Deposit of $12000.00
[Analytics] Metric recorded: Event [Deposit] for $12000.00

Step 6: Customer re-enables Mobile App notifications in settings...
(Attaching MobileAppObserver back to BankAccount)

Step 7: Withdrawing $500.00. Expecting notifications to UI, Logger, Fraud AI...

[BankAccount] Withdrew $500.00. New balance: $12750.00
[UI Observer] Balance display updated to $12750.00
[Logger Observer] Transaction logged: Withdrawal of $500.00, new balance: $12750.00
[Fraud AI] Scan complete: Withdrawal of $500.00 is marked OK.
[Mobile App] Push: Account debited with $500.00
[Analytics] Metric recorded: Event [Withdrawal] for $500.00

Step 8: Attempting to withdraw $15000.00...
(Expecting Insufficient Funds failure alerts to remaining observers)

[BankAccount] Rejected: Insufficient funds for withdrawal of $15000.00
[UI Observer] Balance display updated to $12750.00
[Logger Observer] Transaction logged: FailedWithdrawal of $15000.00, new balance: $12750.00
[Fraud AI] WARNING: Suspicious large activity detected: FailedWithdrawal of $15000.00
[Mobile App] Alert: Withdrawal of $15000.00 failed (Insufficient Funds).
[Analytics] Metric recorded: Event [FailedWithdrawal] for $15000.00

=== PATTERN DEMO COMPLETED ===
=====

```

Terminal — bank_observer_pattern_demo (part 2)

5

Deposit \$12 000 — remaining observers only,
Fraud AI warns

6

Customer re-enables Mobile App in settings

7

Withdraw \$500 — UI, Logger and Fraud AI react

8

Withdraw \$15 000 — rejected, failure
broadcast

Observers come and go; BankAccount never
notifies.

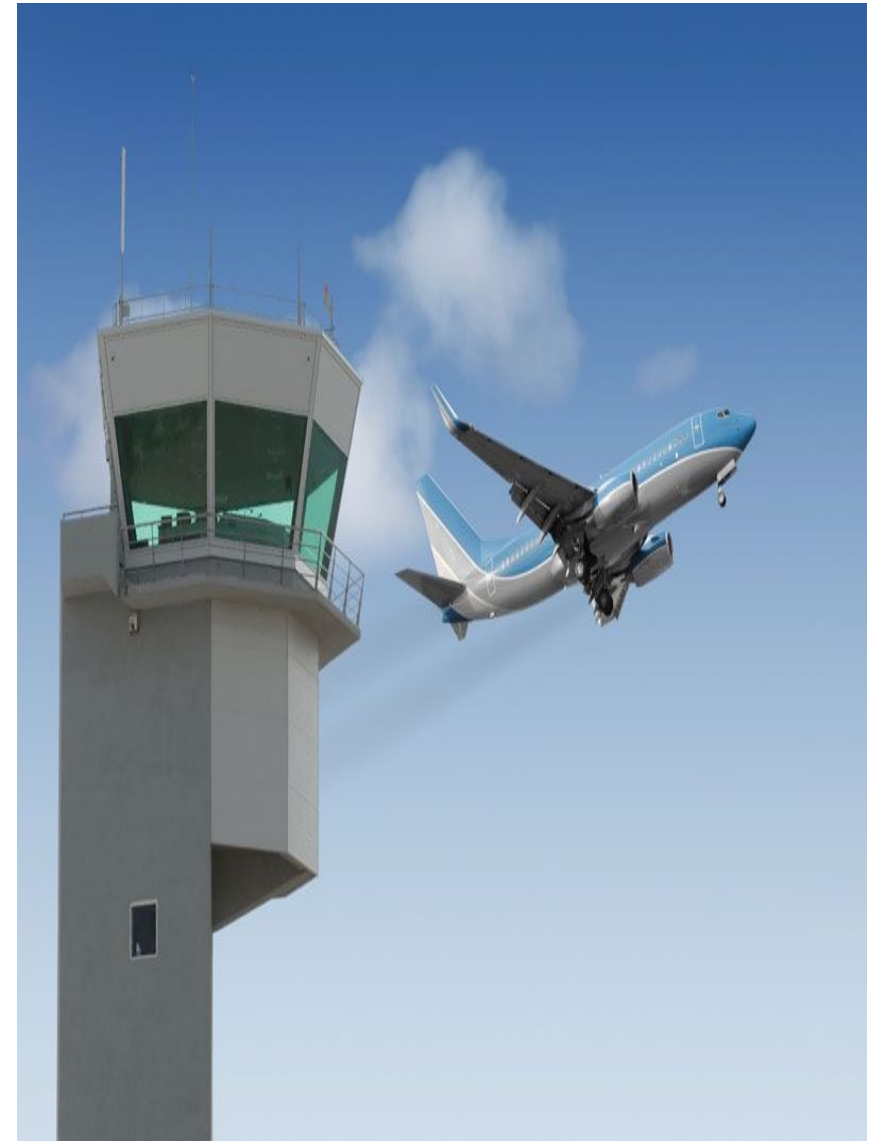
RELATED PATTERN

The Mediator pattern

A behavioural pattern that reduces chaotic dependencies between objects.

It restricts direct communication between objects — the colleagues — and forces them to collaborate exclusively through one centralised mediator object.

Airplanes communicate solely with the control tower — the mediator — which safely orchestrates all the traffic.



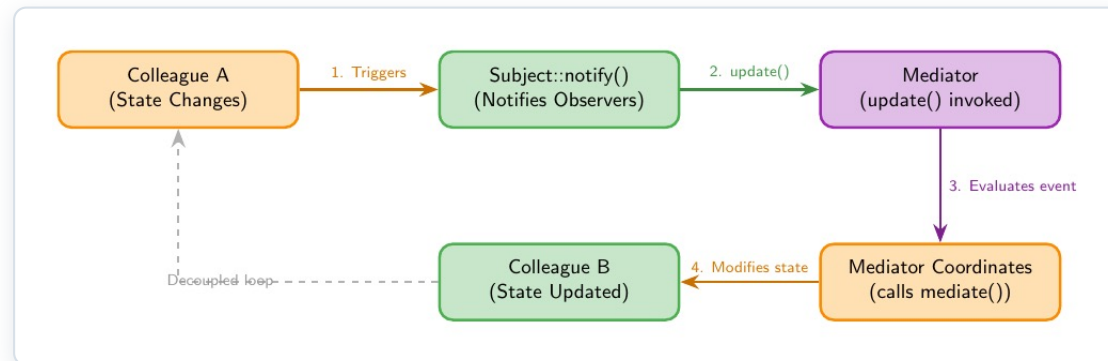
FUSION

Observer + Mediator, fused

Colleagues act as Subjects

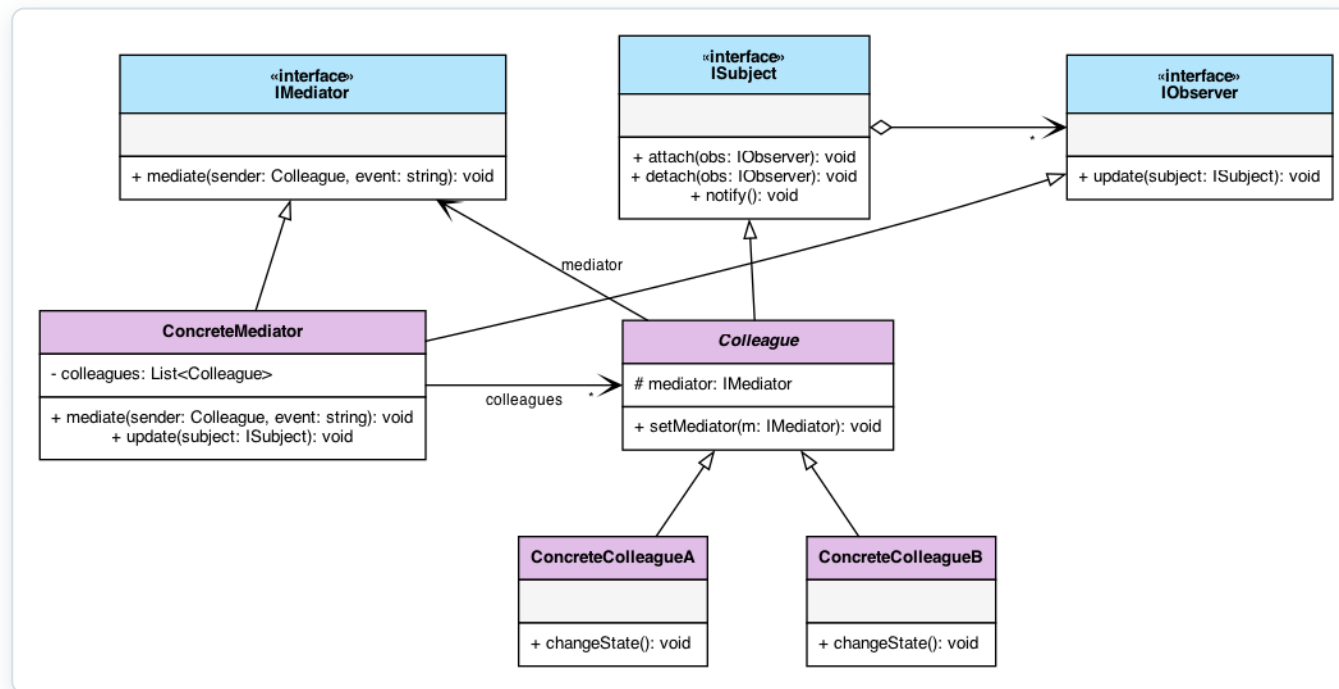
The Mediator acts as the Observer

When a colleague's state changes it raises an event. The mediator — subscribed to those events — catches the notification, evaluates the global context, and updates the other relevant colleagues.



Colleague A changes → Subject::notify() → Mediator evaluates → Colleague B updated

Observer–Mediator class diagram



ConcreteMediator implements IObserver and is notified by every colleague it coordinates

IN THE WILD

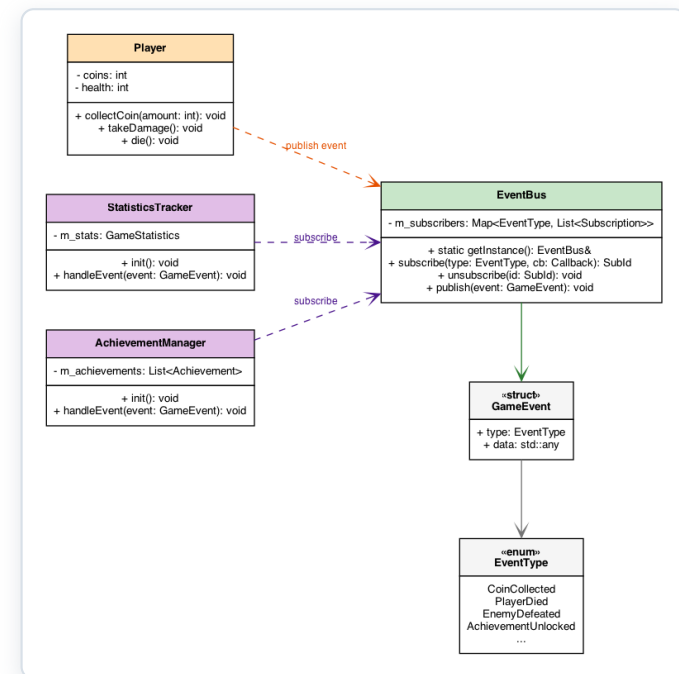
EventBus in game development

A player character takes damage and dies — it is the event publisher.

The player does not call AchievementManager or StatisticsTracker. It simply publishes a PlayerDied event to the central EventBus.

Every system that subscribed to that event type is notified immediately and runs its own logic.

Add a new system — quests, analytics, telemetry — by subscribing to PlayerDied. The player code never changes.



RELATED PATTERN

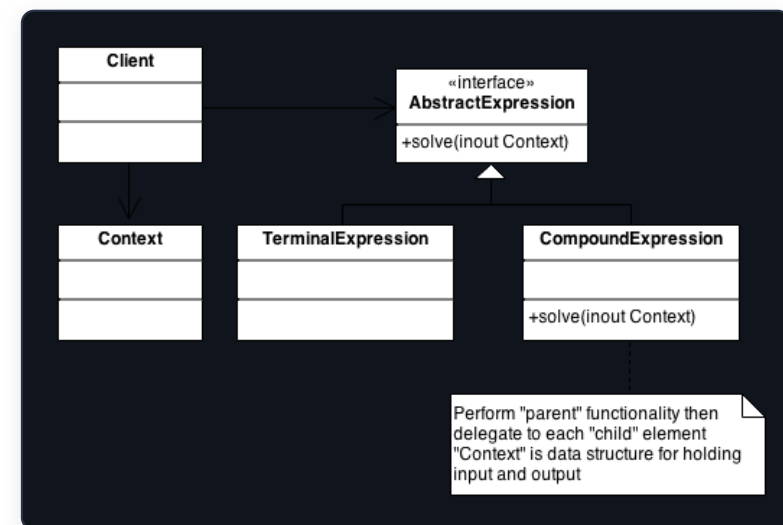
The Interpreter pattern

Given a language, we need a hierarchical representation for its expressions.

Each expression node knows how to solve itself and delegates to its children, so a sentence in the language becomes a tree of objects.

EXAMPLE

$(5 + 1) \times 10 + 41 - x$ as an operation tree



Source: sourcemaking.com

When a leaf can change

If a terminal expression is a variable — something another process or service can change — instead of a constant, the results computed above it are no longer valid.

01

The variable changes

Some other service writes a new value into the terminal node.



02

The observer is notified

The expression node subscribed to that variable receives `update()`.



03

The tree re-evaluates

Only the affected branch recomputes — the rest of the tree is untouched.

⇒ **Observer watches the variable; the Interpreter tree keeps its results in sync.**

USAGE WITH INTERPRETER

A spreadsheet, concretely

In a spreadsheet, cell A3 holds the formula “= A1 + A2”. Its result depends on A1 and A2, which can be rewritten by another formula or by the user at any moment.

INTERPRETER

Parses the formula in A3 into an expression tree.

OBSERVER

Watches A1 and A2 and re-evaluates on change.

A	
1	1200.00
2	450.00
3	= A1 + A2 → 1650.00

A3 subscribes to A1 and A2. Change either one and A3 re-interprets its formula immediately — no polling, and no recalculation of the whole sheet.

EVALUATION

Pros



Open/Closed Principle

New observer classes can be introduced without modifying the subject's code at all.



Loose coupling

The subject interacts only with a generic interface, staying ignorant of every concrete observer implementation.



Dynamic relationships

Connections between publishers and subscribers can be established or terminated at runtime.

The subject only ever talks to an interface — that single decision buys all three.

Cons



The lapsed listener problem

If an observer is never detached, the subject keeps a reference to it — a dangling pointer and a fatal crash when it notifies deleted memory.



Non-deterministic order

The order in which subscribers are notified is arbitrary and cannot be relied on for sequential logic.



Harder debugging

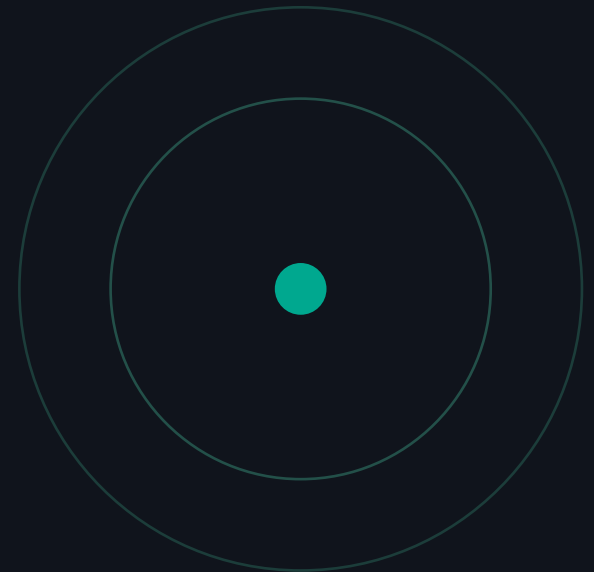
Tracing execution through a long chain of dynamic, event-driven updates is difficult to step through in a debugger.

None of these are fatal — but they are the price of decoupling, and they must be designed for.

CHECK YOURSELF

Quiz

Eight questions on the Observer pattern



QUESTION 1 OF 8

If the Observer pattern were a real-world subscription service, which of the following scenarios best describes it?

- A** A client polling a server every 5 seconds to check if there are new emails.
- B** A newspaper publisher automatically delivering the daily issue to all registered home addresses.
- C** A customer walking into a physical store every day to check if a shoe is back in stock.
- D** A mediator forwarding calls between two people who don't know each other.

If the Observer pattern were a real-world subscription service, which of the following scenarios best describes it?

A A client polling a server every 5 seconds to check if there are new emails.

B A newspaper publisher automatically delivering the daily issue to all registered home addresses. ✓

C A customer walking into a physical store every day to check if a shoe is back in stock.

D A mediator forwarding calls between two people who don't know each other.

Correct answer: B

QUESTION 2 OF 8

In our naïve Bank Account implementation, why did we violate the Open–Closed Principle (OCP)?

- A** Because the BankAccount class was too small to hold the logic.
- B** Because adding a new service required modifying the existing code of BankAccount.
- C** Because we used C++ smart pointers instead of raw pointers.
- D** Because the deposit() function was private.

In our naïve Bank Account implementation, why did we violate the Open–Closed Principle (OCP)?

- A** Because the BankAccount class was too small to hold the logic.
- B** Because adding a new service required modifying the existing code of BankAccount. ✓
- C** Because we used C++ smart pointers instead of raw pointers.
- D** Because the deposit() function was private.

Correct answer: B

QUESTION 3 OF 8

A temporary UI window (observer) is deleted in C++ but forgets to call detach() on the BankAccount. What is the likely runtime result of the next deposit?

- A** The compiler will fail with a syntax error.
- B** The program will run faster because there's one less service.
- C** The program will crash or experience undefined behaviour (dangling pointer).
- D** The bank account will automatically recreate the UI window.

ANSWER · QUESTION 3 OF 8

A temporary UI window (observer) is deleted in C++ but forgets to call detach() on the BankAccount. What is the likely runtime result of the next deposit?

- A** The compiler will fail with a syntax error.
- B** The program will run faster because there's one less service.
- C** The program will crash or experience undefined behaviour (dangling pointer). ✓
- D** The bank account will automatically recreate the UI window.

Correct answer: C

QUESTION 4 OF 8

If observers must be notified in a strict order (e.g. Security Logger before UI updates), is the standard Observer pattern a good choice?

- A** Yes, because the standard Observer pattern guarantees alphabetical notification order.
- B** Yes, because observers are always notified in the exact order they subscribed.
- C** No, because the Observer pattern is only meant for web browsers.
- D** No, because the standard pattern makes no assumptions about notification order, and enforcing it violates loose coupling.

ANSWER · QUESTION 4 OF 8

If observers must be notified in a strict order (e.g. Security Logger before UI updates), is the standard Observer pattern a good choice?

- A** Yes, because the standard Observer pattern guarantees alphabetical notification order.
- B** Yes, because observers are always notified in the exact order they subscribed.
- C** No, because the Observer pattern is only meant for web browsers.
- D** No, because the standard pattern makes no assumptions about notification order, and enforcing it violates loose coupling. ✓

Correct answer: D

QUESTION 5 OF 8

In modern C++, if a Subject holds strong `std::shared_ptr<Observer>` references to its observers, and those observers hold `std::shared_ptr<Subject>` back, what issue arises?

- A** The compiler throws a syntax error.
- B** A circular reference occurs, preventing objects from ever being destroyed (memory leak).
- C** The program automatically converts them into raw pointers.
- D** Observers are executed in reverse order.

ANSWER · QUESTION 5 OF 8

In modern C++, if a Subject holds strong `std::shared_ptr<Observer>` references to its observers, and those observers hold `std::shared_ptr<Subject>` back, what issue arises?

A The compiler throws a syntax error.

B A circular reference occurs, preventing objects from ever being destroyed (memory leak).



C The program automatically converts them into raw pointers.

D Observers are executed in reverse order.

Correct answer: B

QUESTION 6 OF 8

What dangerous side effect occurs if an observer's update() mutates the subject's state and triggers another deposit() without guard conditions?

A Infinite recursive notification loop leading to a stack overflow crash.

B The operating system increases process priority.

C The balance doubles on each iteration automatically.

D The observer is automatically removed from the subscriber list.

ANSWER · QUESTION 6 OF 8

What dangerous side effect occurs if an observer's update() mutates the subject's state and triggers another deposit() without guard conditions?

A

Infinite recursive notification loop leading to a stack overflow crash.



B

The operating system increases process priority.

C

The balance doubles on each iteration automatically.

D

The observer is automatically removed from the subscriber list.

Correct answer: A

QUESTION 7 OF 8

How can the Interpreter pattern and the Observer pattern work together in an automated financial trading system?

A

Interpreter parses custom trading rules (e.g. price > 100 AND volume > 500), and Observer broadcasts price updates that trigger the rule evaluation.

B

They cannot be used in the same project.

C

Observer compiles C++ code, while Interpreter runs the executable.

D

Interpreter deletes the Observer's memory pointers.

How can the Interpreter pattern and the Observer pattern work together in an automated financial trading system?

A

Interpreter parses custom trading rules (e.g. price > 100 AND volume > 500), and Observer broadcasts price updates that trigger the rule evaluation.



B

They cannot be used in the same project.

C

Observer compiles C++ code, while Interpreter runs the executable.

D

Interpreter deletes the Observer's memory pointers.

Correct answer: A

QUESTION 8 OF 8

What happens if a concrete observer calls `subject->detach(this)` inside its own `update()` callback while `notify()` is running a simple range-for loop?

- A** The loop safely skips the detached observer.
- B** The C++ compiler creates a vector copy automatically.
- C** The program prompts the user for manual confirmation.
- D** Iterator invalidation / undefined behaviour — modifying a `std::vector` during range-based iteration corrupts the loop.

ANSWER · QUESTION 8 OF 8

What happens if a concrete observer calls `subject->detach(this)` inside its own `update()` callback while `notify()` is running a simple range-for loop?

- A** The loop safely skips the detached observer.
- B** The C++ compiler creates a vector copy automatically.
- C** The program prompts the user for manual confirmation.

D Iterator invalidation / undefined behaviour — modifying a `std::vector` during range-based iteration corrupts the loop.



Correct answer: D

YOUR TURN

Real-world problem: Moodle notifications

The learning management system (LMS) Moodle needs a notification system to deliver announcements and updates to its students.

Thousands of students, dozens of courses, and a growing list of things worth telling them about.

YOUR TASK

Design the notification system for Moodle so that it supports a wide variety of notifications — and so that adding the next kind costs nothing.

1

Announcements

2

New classes

3

New tasks

4

Quizzes

SOURCES

References

1

E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

2

A. Shvets, “Observer Design Pattern,” *Refactoring.Guru*. [Online]. Available: <https://refactoring.guru/design-patterns/observer>

Code, LaTeX report and this deck: [DesignPatternsGroup52](#) · [Observer](#)

GROUP 52 · APCS 2025

Thanks for watching

Questions?

Nguyễn Đình Minh Huy — 25125083

Trần Gia Huy — 25125084

